

V. Diagramme d'états-transitions

1) Définition et objectifs

Un diagramme d'états-transitions permet de décrire le comportement séquentiel d'un objet d'une classe (ou d'un composant logiciel) à l'aide d'états et de transitions. Chaque diagramme d'états-transitions représente les différents états (situations) dans lesquels peut se trouver l'objet ainsi que la façon dont ce dernier passe d'un état à l'autre en réponse à des événements.

De manière générale, un diagramme d'états-transition est toujours associé à une seule classe du système. Il décrit le cycle de vie d'un objet d'une classe en utilisant :

- des **états** (*qui peuvent être pris par l'objet*)
- des **événements** (*qui provoquent la transition d'un état à un autre*)
- des **actions** (*subies ou provoquées qui accompagnent un changement d'état*)
- des **activités** (*qui surviennent tant que l'objet est dans un état donné*)

Etudier les états d'un objet permet de comprendre son comportement dans le système, surtout quand il s'agit d'objets de classes qui ont des traitements complexes. Le comportement d'un objet (*son changement d'états*) dans un diagramme d'états-transitions peut regrouper un ensemble de scénarios de use case où il intervient, on obtient ainsi une vue synthétique de la dynamique de l'objet dans le système.

2) Eléments d'un diagramme d'états-transitions

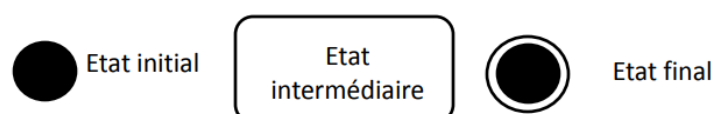
- **Etat :**

Un état est une étape dans le cycle de vie de l'objet. Chaque objet peut passer par une série d'états pendant sa durée de vie. Un état représente une période dans la vie d'un objet pendant laquelle ce dernier attend un événement ou accomplit une activité.

Caractéristiques d'un état :

- Chaque état (intermédiaire) est identifié par un nom
- Un état est stable et possède une certaine durée
- L'ensemble des états du cycle de vie d'un objet est déclenché par un état initial (*correspond à l'état de l'objet juste après sa création, défini par l'un des constructeurs de l'objet*)
- Cet ensemble d'états peut avoir un ou plusieurs états finaux (*correspondent à une phase de destruction de l'objet*). Il peut arriver qu'il n'y ait pas d'état final car un objet peut ne jamais être détruit

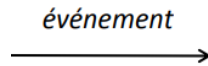
Notations :



- Événement :

Un événement est un fait qui déclenche le changement d'état. Quand un événement est reçu, l'objet bascule dans un nouvel état (désactivation d'un état et activation de l'état suivant). Les événements figurent sur des transitions.

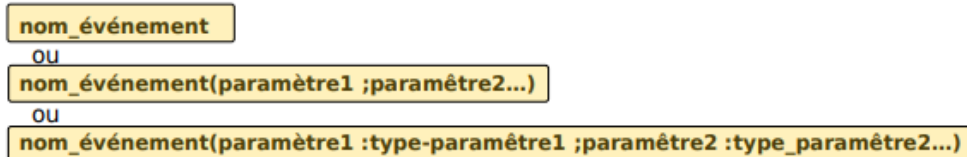
Notation :



Il existe 4 types d'événements :

- **Évènement de type signal (*signal event*)** : il s'agit de la réception d'un message asynchrone (*émit par un autre objet ou par un acteur*).

Syntaxe :



- **Évènement d'appel d'opération (*call event*)** : il s'agit d'un message synchrone (*lié aux cas d'utilisation, opérations du diagramme de classes,...*). Il a la même syntaxe que le précédent
- **Évènement de changement (*change event*)** : il s'agit de la satisfaction d'une condition booléenne. L'expression est testée en permanence jusqu'à ce qu'elle soit vraie pour que la transition se déclenche.

Syntaxe : **when (condition)**

- **Évènement temporel (*time event*)** : il est généré par l'écoulement du temps. Il peut être spécifié de façon absolue (*date précise*) ou de façon relative (*temps écoulé*).

Syntaxe :

Façon absolue : **when(date = date)**

Façon relative : **after(durée)**

- Actions et activités :

Une action est une réaction du système à un événement. Elle se produit quand un événement est déclenché et qu'une condition de garde est vérifiée. *Exemples d'actions* :

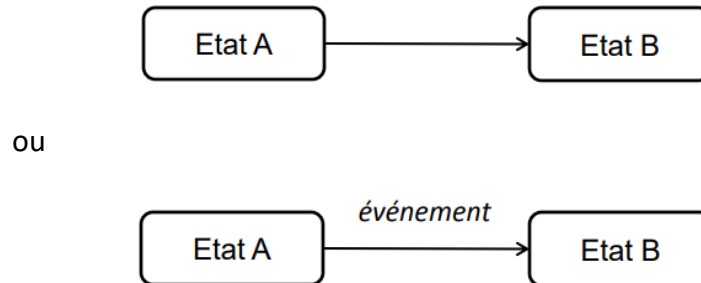
- *affecter une valeur à un attribut,*
- *créer ou détruire un objet,*
- *effectuer une opération (lancer une méthode de classe),*
- *envoyer un signal à un autre objet (ou à soi-même),*
- ...

Une activité est une série d'actions.

- **Transition :**

Les états sont reliés par des transitions qui indiquent qu'un objet se trouvant dans un état peut transiter vers un autre état.

Notation :



Une transition n'a pas de durée, elle est franchie instantanément : on dit qu'elle est déclenchée.

Une transition peut être déclenchée par un événement, elle porte dans ce cas-là le nom de l'évènement (*comme décrit précédemment*). Elle peut aussi être automatique (dans ce cas elle ne porte pas de nom d'évènement).

Il existe deux types de transition : **externe** et **interne**

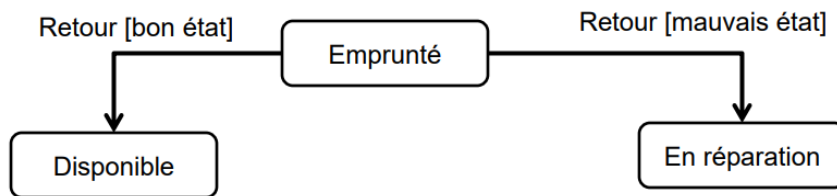
- **Transition externe :** il s'agit du type le plus répandu et utilisé dans les diagrammes d'états-transitions. Celle-ci peut comporter uniquement l'évènement qui la déclenche (tel que dans la notation ci-haut), ou une syntaxe plus évoluée sous la forme suivante : **event[cond]/effect**
 - **event** : il s'agit de l'évènement qui déclenche la transition
 - **cond** : il s'agit de la condition de garde (*expression booléenne*). Pour que la transition soit franchie, il faut que la condition de garde soit remplie en plus de la réception de l'évènement associé. La condition de garde est évaluée uniquement lorsque l'évènement déclencheur se produit (*contrairement à celle de l'évènement de changement « change event » qui est testée en permanence*).
 - **effect** : spécifie une action ou une activité à effectuer lors du déclenchement de la transition

A noter que chacun de ces éléments est facultatif.

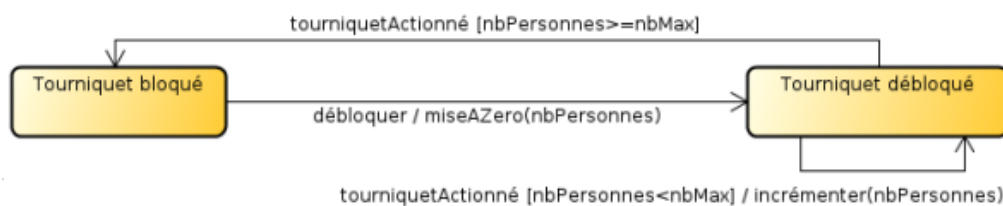
Notation :



Exemple 1 : Etat d'un objet électronique qui a été emprunté



Exemple 2 : Cas d'un tourniquet qui est actionné pour laisser passer à chaque fois un certain nombre de personne uniquement



Remarque : Une transition peut être réflexive.

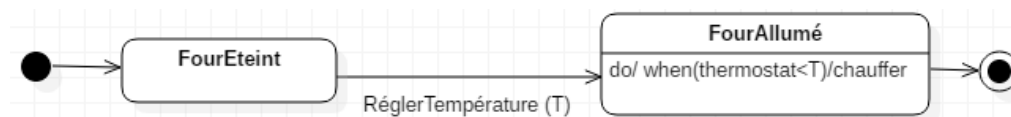
- **Transition interne :** il est possible d'avoir un état qui effectue plusieurs activités successivement ou en parallèle pour lui-même. L'enchaînement de ces activités à l'intérieur d'un même état est spécifié grâce à des transitions internes.

La syntaxe d'une transition interne est la même que celle d'une transition externe. Cependant, les transitions internes s'écrivent à l'intérieur de l'état.

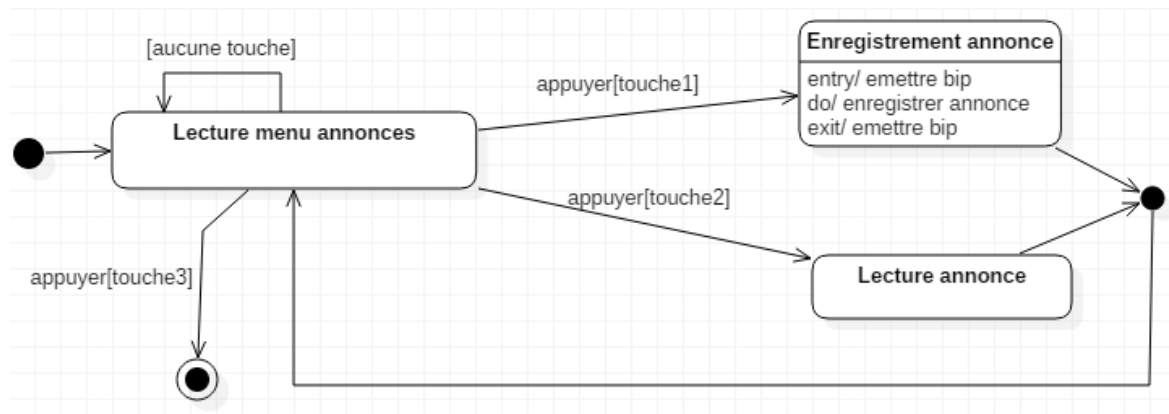
Pour normaliser l'écriture des transitions internes, UML définit des mots clés correspondants à des événements particuliers :

- **entry :** définit l'activité à exécuter lors de l'entrée dans l'état (*entry* est facultatif, mais quand il est dans une transition interne il est toujours suivi de **do**)
- **do :** définit l'activité à exécuter dès que celle définie par **entry** est terminée
- **exit :** définit l'activité à exécuter lors de la sortie de l'état (*facultatif*)

Exemple 1 : Etat d'un four



Exemple 2 : Enregistrement de l'annonce d'accueil d'un répondeur téléphonique

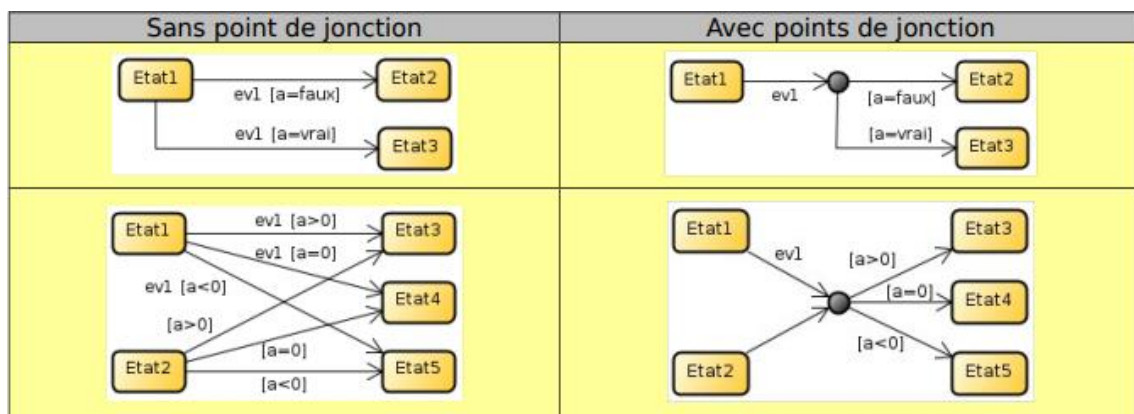


- Point de jonction :

Les points de jonctions permettent à plusieurs transitions d'avoir une partie commune en partageant des segments de transition (comme dans l'exemple précédent). L'utilisation de points de jonction a pour but de rendre la notation des transitions alternatives plus lisible (son utilisation est donc facultative)

Notation : ●

Exemples de comparaison:



Remarque : notons qu'un point de jonction sert avant tout à la factorisation mais il sert aussi à représenter un branchement conditionnel pour un même événement déclenché mais avec des conditions de gardes différentes.

- Point de décision :

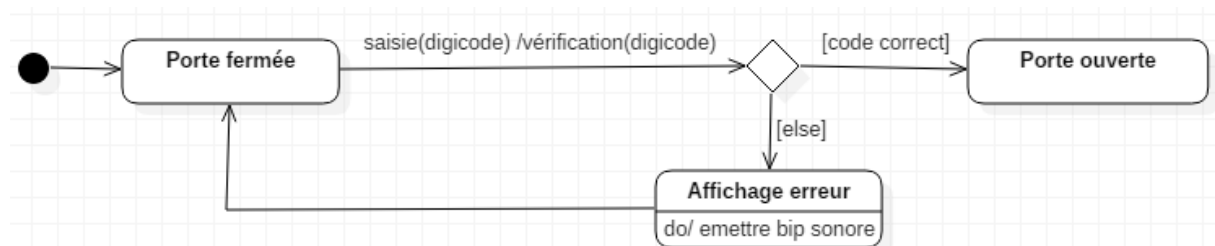
Un point de décision possède une entrée et au moins deux sorties. Il a un fonctionnement similaire à celui du point de jonction. Parmi les différences :

- les conditions de gardes situées après le point de décision sont évaluées au moment où il est atteint par l'évènement (et par l'activité ou l'action qui lui est associée). Il est ainsi possible de sortir de l'état source dès que l'évènement qui conduit au point de décision est déclenché
- lorsque nous arrivons à un point de décision, il faut qu'un des segments de transition qui suit soit franchissable (car nous avons à ce moment-là quitter l'état source de l'objet), il faut donc prévoir le cas où aucune transition n'est franchissable grâce à l'utilisation d'une clause **[else]**. Celle-ci est recommandée après un point de décision, car elle garantit un modèle bien formé.

A noter qu'un segment de transition franchi après un point de décision ne peut pas comporter d'évènement.

Notation : 

Exemple : Cas de l'ouverture d'une porte avec un code d'accès (digicode)



- Etat composite :

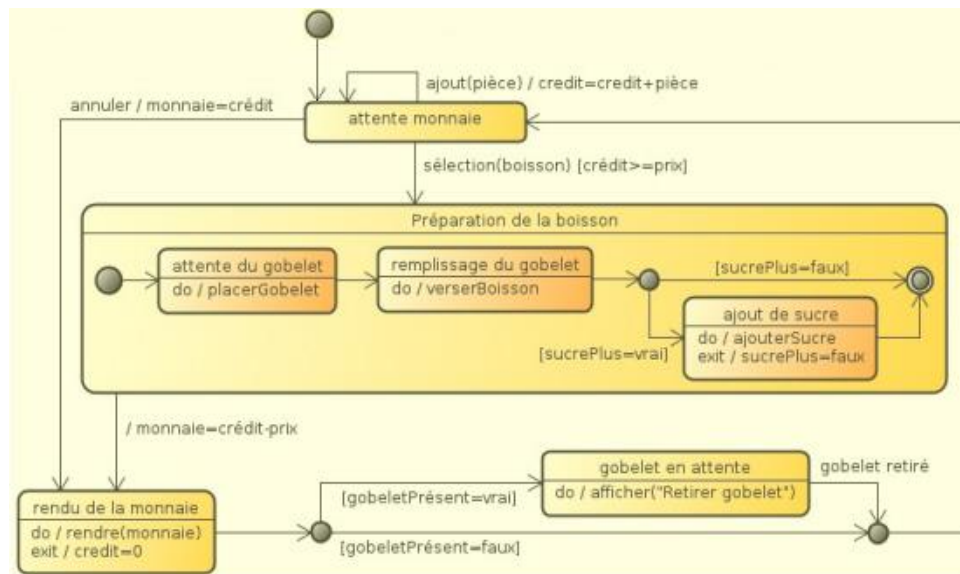
Certains états sont complexes et correspondent à la réalisation de plusieurs activités (*successivement ou en parallèle*) qui ne pourront pas être définis par des transitions internes. On peut alors les décomposer en sous-états. C'est ce qu'on appelle des états composites.

Un état composite est un état décomposé en régions contenant chacune un ou plusieurs sous-états. Il permet de structurer les comportements complexes et factoriser les activités.

Notation :



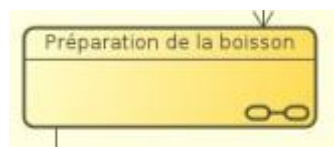
Exemple : cas d'un distributeur automatique de boisson



Remarque 1 :

Pour alléger la présentation d'un diagramme d'états-transitions, on peut faire référence à l'état composite en utilisant sa notation abrégée. Il sera ainsi défini ailleurs (dans un autre diagramme).

Exemple : état composite de l'exemple précédent abrégé :

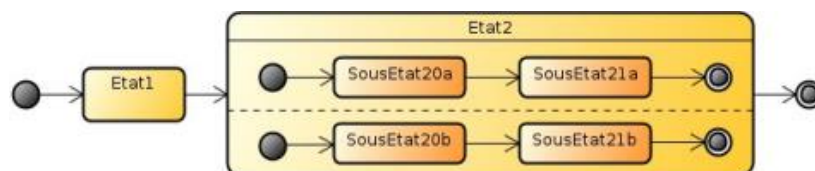


Remarque 2 : Cas de l'état composite orthogonal

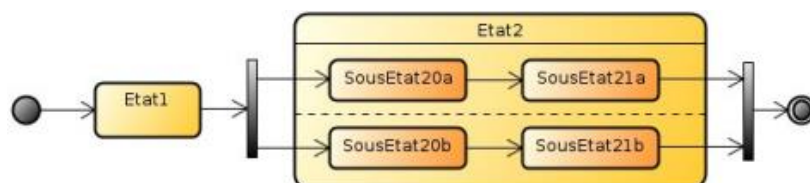
Quand un état composite comporte plus d'une région, il est qualifié d'état orthogonal. Lorsqu'un état orthogonal est actif, un sous-état direct de chaque région est simultanément actif, il y a donc concurrence (*parallélisme*).

Un état composite ne comportant qu'une région est qualifié d'état non orthogonal.

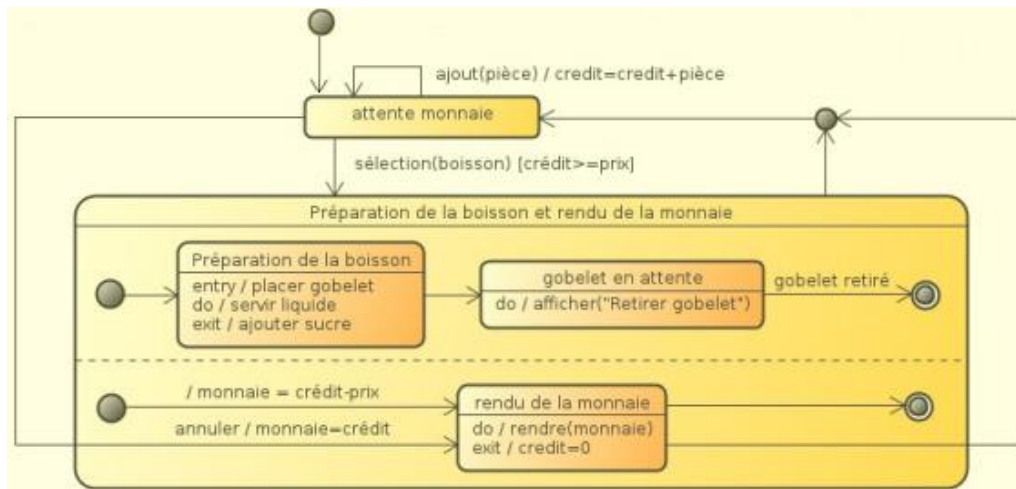
Notation :



On peut également utiliser cette notation avec des barres **fork** et **join** (comme dans les diagrammes d'activités)



Exemple : autre représentation du distributeur automatique de boisson



Remarque générale:

On peut remarquer après toutes ces définitions que les diagrammes d'états-transitions et les diagrammes d'activités ont des définitions et des notations similaires de leur éléments constitutifs, en réalité c'est parce que le diagramme d'activités est dérivé du diagramme d'états-transitions. Cependant, ils ne répondent pas aux mêmes objectifs. Le diagramme d'états-transitions permet d'analyser et de comprendre le comportement des objets du système, alors que le diagramme d'activités est destiné à comprendre le fonctionnement du système de manière général ou d'un processus du système.

3) Exemple d'application :

On souhaite représenter le fonctionnement du système d'alarme d'un magasin.

Pour activer le système d'alarme, le propriétaire des lieux saisit et valide le code 4231. Il peut annuler la saisie. S'il valide le bon code le système s'active. Si le code n'est pas le bon le système reste désactivé.

Pour désactiver le système d'alarme, le propriétaire saisit et valide le même code. Il peut aussi annuler la saisie. S'il valide un mauvais code, il a droit à trois autres tentatives de saisie. S'il échoue à la 4ème tentative, le système d'alarme se déclenche et devient en mode alerte. Si le code est bon le système est désactivé.

Voici un exemple de diagramme d'états-transitions décrivant le cycle de vie du système d'alarme.

